

PROJET 80:

IA ET GPU POUR L'EMBARQUÉ

BOUHALLA Moïse
CHERON Adele
COANON Romain

Encadrant: ABIB Ghalid
2025-2026



IP PARIS

Sommaire

I-Introduction	3
II-Contexte et état de l'art	4
II.1. Intelligence artificielle embarquée	4
II.2. Consommation énergétique des réseaux de neurones	4
II.3. Contraintes des systèmes embarqués	5
II.4 Techniques d'optimisation énergétique	5
○ Quantification	
○ Pruning	
○ Optimisation de l'architecture du réseau de neurones	
III-Problématique et objectifs du projet	7
IV-Méthodologie	8
IV.1 Environnement matériel	8
IV.2 Environnement logiciel	8
IV.3 Métriques étudiées	8
IV.4 Protocoles expérimentaux	9
V-Expérimentations	10
V.1 Quantification	10
V.2 Pruning	11
V.3 Optimisation de l'architecture du réseau de neurones	12
V.4 Analyse de la consommation énergétique	13
V.5 Tests sur modèle d'IA embarqué	13
VI-Résultats expérimentaux	15
VI.1 Résultats individuels	15
VI.2 Résultats du groupe	19
VI.3 Consommation énergétique	20
VI.4 Impact sur les performances	21
VI.5 Analyse et interprétation des résultats	22
VII-Limites et difficultés rencontrées	24
VIII-Perspectives d'amélioration	25
IX-Conclusion	26
X-Annexes	27
XI-Bibliographie	34

I- Introduction

L'intelligence artificielle occupe aujourd'hui une place importante dans de nombreux domaines tels que la robotique, les véhicules autonomes, les drones ou encore les objets connectés. Les progrès récents du deep learning ont permis le développement de modèles toujours plus performants, mais également de plus en plus coûteux en ressources de calcul et en consommation énergétique.

Cette problématique est particulièrement importante dans le cadre des systèmes embarqués, où les capacités matérielles, la mémoire et l'autonomie énergétique restent limitées. Il devient alors nécessaire de développer des solutions permettant de réduire le coût énergétique des modèles d'intelligence artificielle tout en conservant des performances satisfaisantes.

Dans le cadre du projet Cassiopée, nous nous intéressons à différentes méthodes d'optimisation visant à améliorer l'efficacité énergétique des modèles d'IA embarqués. Plusieurs approches seront étudiées, notamment la quantification, le pruning ainsi que l'optimisation de l'architecture des réseaux de neurones.

L'objectif de ce projet est d'analyser l'impact de ces différentes techniques sur la consommation énergétique, les performances ainsi que le temps d'exécution des modèles étudiés. Les expérimentations réalisées permettront de comparer les approches mises en œuvre et d'identifier les compromis possibles entre efficacité énergétique et précision des modèles.

Dans un second temps, les optimisations développées auront vocation à être testées sur une plateforme embarquée de type Jetson Orin Nano afin d'évaluer leur efficacité dans un contexte réel d'IA embarquée.

II-Contexte et état de l'art

II.1 Intelligence artificielle embarquée

L'intelligence artificielle embarquée, également appelée *Edge AI*, désigne l'exécution de modèles d'intelligence artificielle directement sur des appareils embarqués tels que des drones, robots, véhicules autonomes, smartphones ou objets connectés. Contrairement aux approches classiques reposant sur des serveurs distants ou le cloud, les calculs sont réalisés localement sur le dispositif.

Cette approche présente plusieurs avantages, notamment une réduction de la latence, une meilleure confidentialité des données ainsi qu'une plus grande autonomie vis-à-vis d'une connexion réseau. Cependant, les systèmes embarqués disposent généralement de ressources limitées en termes de puissance de calcul, de mémoire et d'énergie.

Les modèles de deep learning modernes étant souvent très coûteux en calcul, leur déploiement sur ce type de matériel représente un défi important. Il devient alors nécessaire d'adapter ou d'optimiser ces modèles afin de réduire leur consommation énergétique et leur complexité tout en conservant des performances satisfaisantes.

Dans ce contexte, de nombreuses techniques d'optimisation ont été développées afin de rendre les modèles d'intelligence artificielle plus adaptés aux contraintes des systèmes embarqués.

II.2 Consommation énergétique des réseaux de neurones

Les réseaux de neurones modernes nécessitent un grand nombre d'opérations mathématiques pour fonctionner, notamment des multiplications matricielles et des convolutions. Ces opérations sont réalisées de manière répétée lors des phases d'entraînement et d'inférence, ce qui entraîne une consommation importante de ressources matérielles et d'énergie.

La consommation énergétique d'un modèle dépend de plusieurs facteurs, tels que la taille du réseau, le nombre de paramètres, le type d'opérations utilisées ou encore le format numérique des données manipulées. Les transferts mémoire et les accès aux données peuvent également représenter une part importante du coût énergétique total.

Cette problématique devient particulièrement critique dans le cadre des systèmes embarqués, où les capacités matérielles et l'autonomie énergétique sont limitées. Une consommation excessive peut réduire l'autonomie d'un appareil, provoquer une surchauffe ou limiter les performances en temps réel.

Afin de limiter ces contraintes, différentes techniques d'optimisation peuvent être utilisées pour réduire le nombre de calculs effectués, diminuer la taille des modèles ou encore simplifier les opérations réalisées par le réseau de neurones.

II.3 Contraintes des systèmes embarqués

Les systèmes embarqués disposent généralement de ressources limitées par rapport aux ordinateurs ou serveurs traditionnels. Les capacités de calcul, la mémoire disponible ainsi que l'autonomie énergétique sont souvent restreintes, ce qui complique l'exécution de modèles d'intelligence artificielle complexes.

Dans de nombreuses applications embarquées, comme les drones, les robots autonomes ou les objets connectés, les modèles doivent fonctionner en temps réel tout en consommant le moins d'énergie possible. Une consommation excessive peut entraîner une diminution de l'autonomie, une augmentation de la température du système ou encore une baisse des performances.

La mémoire constitue également une contrainte importante. Les modèles de deep learning modernes peuvent contenir plusieurs millions de paramètres, nécessitant un espace mémoire conséquent et des transferts de données fréquents. Ces transferts peuvent représenter une part importante du coût énergétique global.

Enfin, les systèmes embarqués doivent souvent trouver un compromis entre précision, rapidité d'exécution et consommation énergétique. Il devient alors nécessaire de développer des techniques d'optimisation permettant d'adapter les modèles d'intelligence artificielle aux contraintes matérielles des plateformes embarquées.

II.4. Technique d'optimisation pour un déploiement sur un système embarqué.

Le déploiement de modèles d'intelligence artificielle sur des systèmes embarqués nécessite l'utilisation de différentes techniques d'optimisation afin de réduire les besoins en calcul, en mémoire et en énergie. Ces méthodes permettent d'adapter les modèles de deep learning aux contraintes matérielles des plateformes embarquées tout en conservant des performances satisfaisantes.

La *quantification* est une première approche couramment utilisée. Elle consiste à réduire la précision numérique des paramètres et des calculs réalisés par le modèle, par exemple en passant du format FP32 vers des formats plus légers comme FP16 ou INT8. Cette technique permet de diminuer l'utilisation mémoire, d'accélérer les calculs et de réduire la consommation énergétique.

Le *pruning* constitue une autre méthode d'optimisation importante. Cette technique vise à supprimer les paramètres ou connexions considérés comme peu utiles dans un réseau de neurones. En réduisant le nombre de calculs nécessaires, le pruning permet d'obtenir des modèles plus légers et potentiellement moins énergivores. Certaines approches utilisent notamment une sparsité structurée afin de faciliter l'exécution sur les architectures matérielles modernes.

L'*optimisation de l'architecture du réseau de neurones* représente également un axe majeur de recherche. Des architectures légères telles que MobileNet ou certaines versions de

YOLO ont été développées afin de faciliter le déploiement sur des systèmes disposant de ressources limitées. Ces modèles cherchent à conserver de bonnes performances tout en réduisant le nombre de paramètres et le coût computationnel.

Enfin, des outils d'optimisation matérielle et logicielle peuvent être utilisés afin d'améliorer les performances des modèles sur des plateformes embarquées. Des technologies comme CUDA ou TensorRT, développées par NVIDIA, permettent notamment d'optimiser l'exécution des réseaux de neurones sur GPU embarqués tels que les Jetson.

III-Problématique et objectifs du projet

Les modèles d'intelligence artificielle utilisés dans les applications modernes sont souvent conçus pour être exécutés sur des infrastructures disposant d'importantes ressources matérielles. Cependant, leur déploiement sur des systèmes embarqués pose plusieurs difficultés liées aux contraintes de calcul, de mémoire et de consommation énergétique.

Dans ce contexte, une question importante apparaît : comment réduire le coût énergétique d'un modèle d'intelligence artificielle tout en conservant des performances suffisantes pour une utilisation embarquée ?

Le projet Cassiopée cherche ainsi à étudier l'impact de différentes techniques d'optimisation sur l'efficacité énergétique des modèles d'IA. Les travaux réalisés porteront notamment sur la quantification, le pruning ainsi que l'optimisation de l'architecture des réseaux de neurones.

Chaque membre du groupe étudiera une approche d'optimisation spécifique afin d'analyser son impact sur les performances et la consommation énergétique des modèles. Les différents résultats obtenus seront ensuite rassemblés afin de réaliser une analyse globale et d'effectuer des calculs énergétiques comparatifs entre les différentes méthodes étudiées.

L'objectif sera de comparer ces approches à travers plusieurs métriques telles que le temps d'exécution, la consommation énergétique, l'utilisation mémoire et les performances du modèle, afin d'identifier les compromis possibles entre efficacité énergétique et précision.

Enfin, les optimisations étudiées auront vocation à être testées sur une plateforme embarquée de type Jetson Orin Nano afin d'évaluer leur pertinence dans un contexte réel d'IA embarquée.

IV- Méthodologie

IV.1. Environnement matériel

Pour mener les expérimentations, nous avons utilisé plusieurs types de supports matériels. Une partie des tests a été réalisée sur des ordinateurs disposant de capacités de calcul importantes, afin de développer, entraîner et comparer les différentes méthodes d'optimisation dans de bonnes conditions.

Dans un second temps, les approches étudiées ont vocation à être testées sur une plateforme embarquée de type Jetson Orin Nano. Cette carte permet d'évaluer les performances des modèles dans un contexte plus proche d'une application réelle embarquée, avec des contraintes de calcul, de mémoire et d'énergie plus importantes.

Une caméra d'acquisition est également utilisée afin de réaliser des tests de détection de visage en temps réel. Cette partie permet d'évaluer le comportement des modèles optimisés dans une situation concrète d'utilisation embarquée.

IV.2. Environnement logiciel

Le développement des différents modèles et méthodes d'optimisation a été réalisé en Python, principalement à l'aide des bibliothèques PyTorch et CuPy. Ces frameworks permettent de construire, entraîner et évaluer des modèles de deep learning tout en facilitant l'utilisation de l'accélération GPU.

Les technologies CUDA et TensorRT, développées par NVIDIA, ont également été utilisées afin d'exploiter les capacités de calcul GPU et d'optimiser les performances des modèles lors de l'inférence. CUDA permet notamment l'exécution parallèle des calculs sur GPU, tandis que TensorRT propose des optimisations adaptées au déploiement de modèles d'intelligence artificielle sur systèmes embarqués.

Le modèle YOLO (*You Only Look Once*) est utilisé comme base commune afin de rassembler et comparer les résultats obtenus par les différentes approches étudiées. YOLO est un modèle de détection d'objets en temps réel particulièrement utilisé dans les applications de vision par ordinateur grâce à sa rapidité d'exécution et à son efficacité. Ce choix permet d'évaluer les optimisations sur un modèle adapté aux contraintes des systèmes embarqués.

IV.3. Métriques étudiées

Afin d'évaluer l'efficacité des différentes optimisations, plusieurs métriques sont étudiées. Le temps d'exécution permet de mesurer la rapidité du modèle lors de l'inférence.

La consommation énergétique est estimée à partir de la puissance délivrée par le processeur ou la carte graphique pendant le temps d'exécution du modèle. Cette approche permet d'obtenir une estimation de l'énergie consommée lors des calculs effectués.

L'utilisation mémoire est également prise en compte, car elle constitue une contrainte importante pour les systèmes embarqués. Enfin, les performances du modèle sont évaluées afin de vérifier que les optimisations proposées ne dégradent pas excessivement la qualité des prédictions.

IV.4 Protocole expérimental

Les expérimentations sont organisées autour d'une comparaison entre un modèle de référence et plusieurs versions optimisées. Chaque approche, comme la quantification, le pruning ou l'optimisation de l'architecture, est étudiée séparément afin d'analyser son impact propre.

Les résultats obtenus par chaque membre du groupe seront ensuite rassemblés afin d'effectuer une analyse globale. Cette mise en commun permettra de comparer les différentes méthodes selon les mêmes critères et de réaliser des calculs énergétiques comparatifs.

Enfin, les optimisations les plus pertinentes pourront être testées sur la Jetson Orin Nano avec la caméra d'acquisition, afin d'évaluer leur comportement dans un contexte réel d'IA embarquée.

V- Expérimentation

V.1 Quantification

La quantification est une technique d'optimisation qui consiste à réduire la précision numérique utilisée pour représenter les données manipulées par un modèle ou par un calcul. Dans le domaine du deep learning, les poids des réseaux de neurones sont généralement stockés sous forme de nombres flottants, le plus souvent en float32, c'est-à-dire avec un encodage sur 32 bits. Il est cependant possible d'utiliser des formats de précision plus faibles, comme le float16, l'int8 ou encore l'int4. Le principe est alors de représenter les mêmes informations avec un nombre de bits plus réduit.

Cette réduction de précision présente plusieurs intérêts. Tout d'abord, elle diminue la taille mémoire nécessaire pour stocker les poids du modèle. Par exemple, un poids encodé en float16 occupe deux fois moins de mémoire qu'un poids encodé en float32, tandis qu'un encodage en int8 permet théoriquement une réduction encore plus importante. Cette diminution de la taille des modèles est particulièrement intéressante dans le cadre de l'IA embarquée, où les ressources mémoire sont limitées.

La quantification peut également permettre de réduire le coût des calculs réalisés lors de l'inférence. En effet, manipuler des données de plus faible précision réduit la quantité de données transférées en mémoire et peut permettre au matériel d'exécuter certaines opérations plus rapidement. Les GPU modernes disposent notamment d'unités de calcul spécialisées capables d'accélérer les opérations effectuées sur des formats réduits comme le float16 ou l'int8. Une réduction du temps de calcul peut alors entraîner une diminution de l'énergie consommée par opération, ce qui constitue un enjeu important dans les systèmes embarqués ou les applications temps réel.

Afin d'étudier ces mécanismes à un niveau bas, une première approche a consisté à analyser l'effet de la quantification sur une opération fondamentale des réseaux de neurones : le produit matriciel. En effet, une grande partie des calculs réalisés dans les couches linéaires ou convolutives des réseaux de neurones repose sur des opérations matricielles. Étudier ce calcul de manière isolée permet donc de mieux comprendre l'impact des différents formats numériques sur les performances.

Pour réaliser ces expérimentations, la bibliothèque CuPy a été utilisée. CuPy propose une interface proche de NumPy, mais permet d'exécuter les calculs directement sur GPU. Plusieurs produits matriciels ont ainsi été réalisés avec des matrices utilisant différents formats numériques, notamment float64, float32 et float16. Les calculs étant exécutés sur GPU, il a été nécessaire de prendre en compte le caractère asynchrone des opérations. En effet, lorsqu'un calcul est lancé sur GPU, le programme Python peut continuer son exécution avant que l'opération ne soit réellement terminée. Afin d'obtenir des mesures de temps correctes, une synchronisation explicite a donc été utilisée (`cp.cuda.Stream.null.synchronize()`), avant l'arrêt du chronomètre.

L'objectif de cette première étude était principalement de comparer le comportement des différents formats flottants et d'observer l'influence de la précision numérique sur les calculs

matriciels. Une attention particulière a également été portée aux formats entiers, notamment l'int8, largement utilisé dans les techniques modernes de quantification des modèles d'intelligence artificielle. Cependant, les outils utilisés dans le cadre de cette expérimentation ne permettaient pas d'exploiter simplement certains calculs matriciels sur des matrices entières dans les mêmes conditions que les formats flottants.

Afin de poursuivre l'étude des formats entiers, une implémentation manuelle du produit matriciel a donc été réalisée en Python. Cette approche permettait de manipuler explicitement des matrices de type entier et de mieux contrôler les opérations effectuées lors du calcul. Bien que cette implémentation ne bénéficie pas des optimisations matérielles avancées proposées par les bibliothèques spécialisées GPU, elle permettait néanmoins d'étudier le principe de la quantification et ses effets sur la représentation des données et les opérations réalisées.

Cette approche bas niveau avait ainsi pour objectif de mieux comprendre les mécanismes fondamentaux liés à la quantification avant d'étudier ces techniques sur des modèles d'intelligence artificielle complets exécutés dans un contexte d'IA embarquée.

V.2 Pruning

Le pruning est une technique d'optimisation qui consiste à mettre à zéro certains coefficients considérés comme peu significatifs, afin de réduire le nombre d'opérations nécessaires lors des calculs. Dans cette partie, l'objectif était d'étudier si cette méthode pouvait permettre de diminuer le temps d'exécution et la consommation énergétique d'un produit matriciel exécuté sur GPU.

Une première approche a consisté à définir une fonction de pruning prenant en argument une matrice et un seuil positif (h). Cette fonction parcourt la matrice et remplace par zéro tous les coefficients dont la valeur absolue est inférieure à (h). Deux matrices (A) et (B) ont ensuite été générées aléatoirement selon une loi normale, à l'aide de la fonction `cp.random.normal`, avec une taille de (10000 $\times$ 10000) et un format `float64`. Le produit matriciel classique entre (A) et (B) a ensuite été comparé au produit matriciel entre les matrices prunées (A_p) et (B_p).

Pour évaluer les performances, le temps d'exécution du programme ainsi que la consommation énergétique ont été étudiés. La puissance délivrée par la carte graphique a été relevée via l'invite de commande, tandis que le temps d'exécution a été mesuré en synchronisant le chronomètre avec le début et la fin du traitement GPU à l'aide de la commande `cp.cuda.Stream.null.synchronize()`. Cette étape est importante, car les calculs GPU sont asynchrones : sans synchronisation, le temps mesuré peut être sous-estimé.

Une deuxième approche a consisté à exploiter explicitement les zéros créés par le pruning. Pour cela, les matrices prunées ont été converties en format creux (*sparse*) afin d'éviter, en théorie, de réaliser des calculs avec des coefficients nuls. La sparsité des matrices a également été mesurée afin de vérifier si cette approche pouvait être pertinente.

Enfin, une troisième approche a consisté à développer une implémentation manuelle du produit matriciel, alors que les premières expérimentations utilisaient directement l'opérateur @. L'objectif était de mieux contrôler les calculs réalisés et d'exploiter explicitement la sparsité générée par le pruning en évitant certaines opérations impliquant des coefficients nuls. Le calcul matriciel codé ne calcule pas les opérations où un des deux coefficients est nul. Cette implémentation a été réalisée avec la bibliothèque PyTorch.

Les différentes approches étudiées ont ensuite été comparées à travers plusieurs métriques, notamment le temps d'exécution, le taux de sparsité obtenu ainsi que la consommation énergétique estimée. Les résultats détaillés et l'analyse des performances obtenues sont présentés dans la partie VI.

V.3 Architecture

L'idée de départ pour la partie optimisation de l'architecture était de travailler directement sur les couches des CNN, en commençant par une architecture faite main avant de basculer sur YOLO. Mon approche sur ce début de projet Cassiopée a été assez expérimentale, avec une progression à tâtons. Mes cours de Deep Learning en Image m'ont donné les bases du codage de CNN, ce qui m'a permis de me lancer.

J'ai d'abord voulu tester la différence de consommation et de temps d'exécution entre un modèle simpliste de segmentation d'image et un modèle plus robuste avec l'ajout de Batch Norm et de Max Pooling layers. Pour la consommation, accéder à celle du CPU est assez simple, mais celle du GPU est bien plus complexe. J'ai commencé par calculer la consommation en moyennant le début et la fin du processus, ce qui fonctionne pour le CPU mais pas pour le GPU. J'ai ensuite fait une moyenne avec des valeurs prises tout au long du processus, ce qui permet d'avoir des résultats environ une fois sur deux avec le GPU. Je n'ai pas réussi à optimiser cette mesure pendant cette phase.

J'ai ensuite pensé à étudier l'impact sur la consommation et la performance de chaque type de couche d'un CNN. Cette approche n'était pas vraiment concluante, car on ne savait pas exactement quelles couches on allait manipuler derrière avec YOLO.

Il était donc temps de passer directement à l'architecture YOLO. La documentation sur leur site est très dense, et il a fallu s'y plonger avant de commencer les expérimentations. Les recherches ont montré qu'il existe plusieurs versions de YOLO, et qu'il fallait choisir celle qui était la plus adaptée pour le Jetson.

Finalement, en partant de YOLO26n, que nous avons déterminée comme la meilleure version actuelle de YOLO pour le Jetson (réduction de 25% de la consommation par rapport à YOLO26 et bien plus performante que les anciennes versions comme YOLOv8n par exemple), j'ai essayé de travailler sur ses couches. Force est de constater qu'une architecture professionnelle et optimisée comme celle-ci est difficile à comprendre. Elle n'est pas spécialement détaillée dans la documentation, alors j'ai décidé de print l'ensemble des couches pour pouvoir les analyser. L'architecture YOLO26n est constituée d'un total de 257 couches, ce qui rend son analyse complexe. La complexité de cette architecture provient de

plusieurs facteurs. Tout d'abord, le nombre élevé de couches (257) rend difficile la compréhension globale du modèle, surtout quand on ne connaît pas encore parfaitement les interactions entre chaque bloc. Ensuite, les modules modulaires comme C3k2, SPPF ou C2PSA sont des composants avancés, conçus pour optimiser les performances et la consommation, mais leur fonctionnement interne n'est pas toujours bien documenté. Enfin, l'optimisation poussée de YOLO26n pour les appareils edge comme le Jetson signifie que chaque couche a été fine-tunée pour un équilibre précis entre rapidité, consommation et précision, ce qui rend toute modification hasardeuse sans une compréhension approfondie.

J'ai donc exploré les couches de type C3k2 et tenté de supprimer certaines d'entre elles, mais sans obtenir de résultat concluant.

V.4 Analyse de la consommation énergétique

L'évaluation de la consommation énergétique constitue un élément essentiel dans le cadre du déploiement de modèles d'intelligence artificielle sur des systèmes embarqués. En effet, ces plateformes disposent généralement de ressources limitées et doivent fonctionner avec une consommation énergétique réduite afin de préserver leur autonomie et leurs performances.

Dans ce projet, l'analyse énergétique a été réalisée en étudiant la puissance consommée par le matériel lors de l'exécution des différents modèles et algorithmes d'optimisation. Le temps d'exécution a été mesuré en synchronisant les calculs GPU grâce à la commande `cp.cuda.Stream.null.synchronize()`, permettant d'obtenir une mesure représentative du temps réellement passé à effectuer les calculs.

L'énergie consommée a ensuite été estimée à partir de la relation :

$$E = P \times t$$

où (P) représente la puissance moyenne délivrée par le matériel et (t) le temps d'exécution du programme. Cette méthode permet d'obtenir une estimation de l'énergie nécessaire à l'exécution des différentes versions des modèles étudiés.

Les mesures réalisées ont ensuite été utilisées pour comparer l'impact des différentes techniques d'optimisation, telles que la quantification, le pruning ou l'utilisation de TensorRT. Une attention particulière a été portée à l'énergie consommée par image lors de l'inférence, cette métrique étant particulièrement pertinente dans le cadre d'applications de vision embarquée en temps réel.

V.5 Tests sur modèle d'IA embarqué

Afin d'évaluer l'impact des différentes optimisations dans un contexte proche d'une utilisation embarquée, des expérimentations ont été réalisées sur le modèle YOLO26n. Ce modèle a été utilisé pour effectuer des inférences sur des images de test, notamment une

image représentant une scène urbaine contenant plusieurs objets à détecter.



Plusieurs versions du modèle ont été comparées, notamment une version exécutée sur CPU ainsi que des versions optimisées à l'aide de TensorRT et de la quantification (FP16 et INT8). L'objectif est d'évaluer l'influence de ces optimisations sur les performances du modèle et sur sa consommation énergétique.

Les mesures réalisées portent principalement sur le temps d'inférence et l'énergie consommée par image. Ces résultats permettent d'identifier les optimisations les plus adaptées à un futur déploiement sur plateforme embarquée.

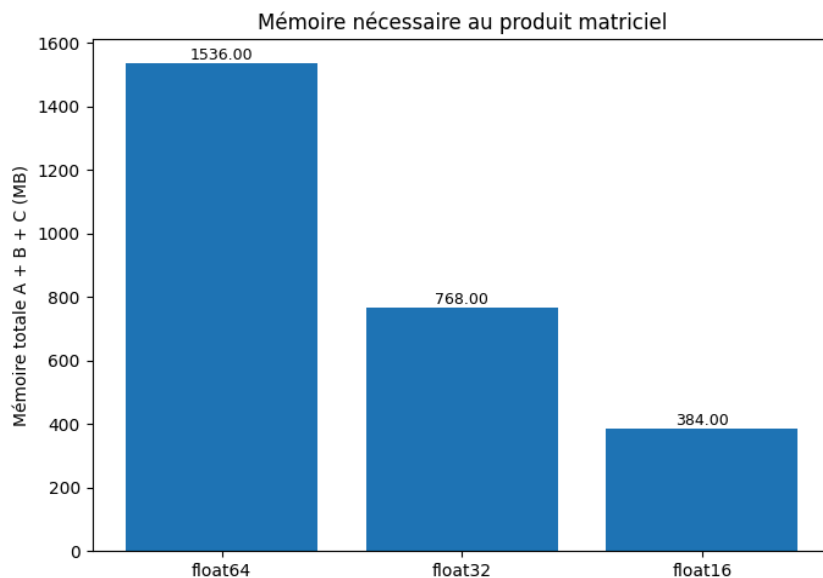
À terme, les optimisations retenues pourront être déployées sur une Jetson Orin Nano et associées à une caméra d'acquisition afin de réaliser des tests en conditions réelles de détection en temps réel.

VI- Résultats expérimentaux

VI.1. Résultats individuels:

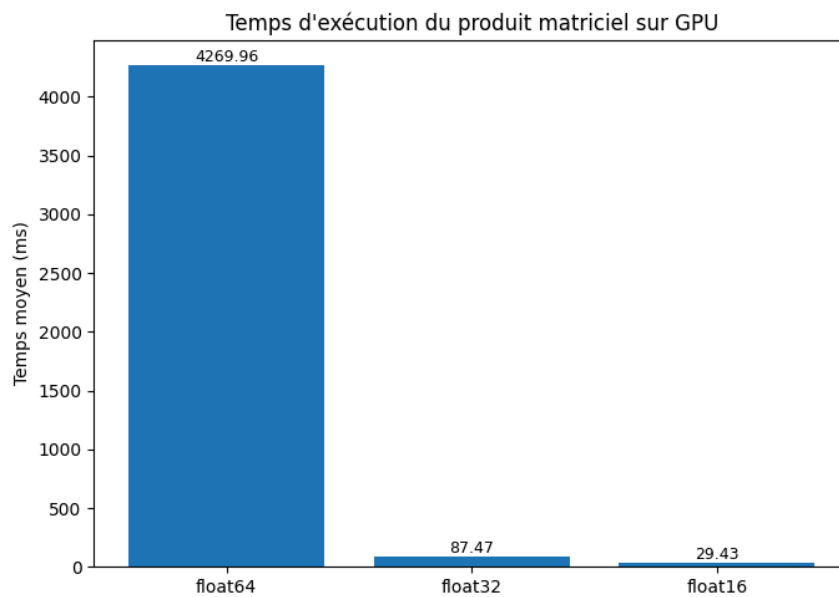
Quantification

Les résultats obtenus montrent clairement l'impact de la précision numérique sur les performances du produit matriciel exécuté sur GPU. La réduction du format des données entraîne d'abord une diminution importante de la mémoire nécessaire pour stocker les matrices. Comme illustré par les graphes, le passage du float64 au float32 divise par deux la mémoire utilisée, tandis que le float16 réduit encore cette taille d'un facteur deux.

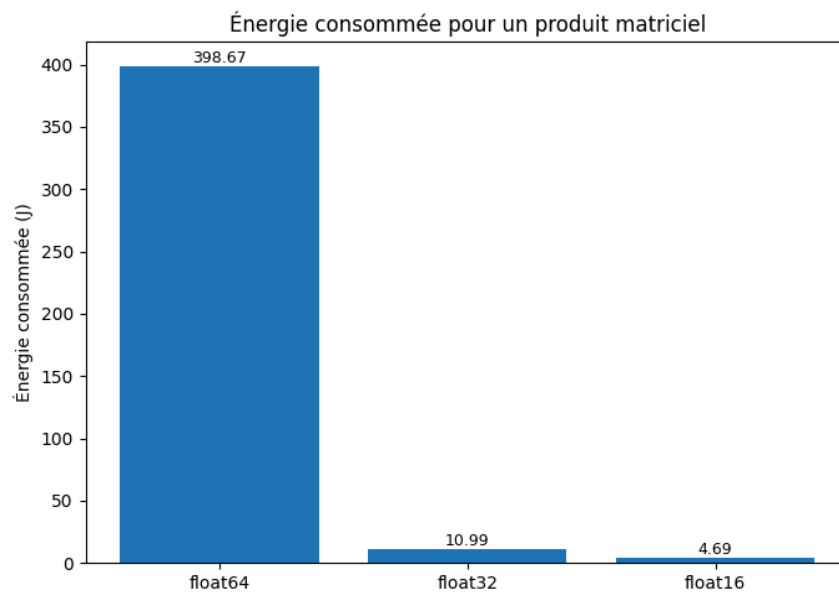


Cette réduction de précision influence également fortement le temps d'exécution. Les calculs réalisés en float16 sont beaucoup plus rapides que ceux effectués en float32 et surtout en float64. Cela s'explique par le fait que les GPU modernes sont fortement optimisés pour les calculs en précision réduite, notamment dans les applications

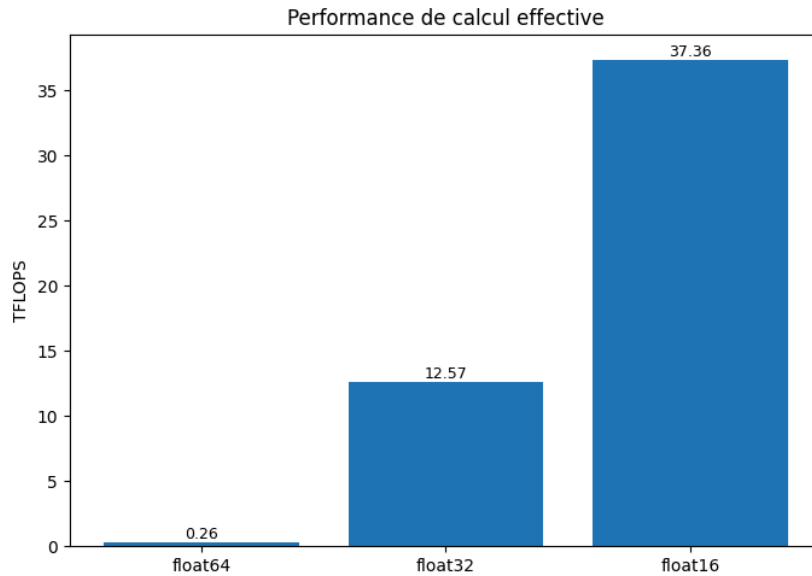
d'intelligence artificielle.



La consommation énergétique suit la même tendance. Malgré une puissance instantanée parfois plus élevée en float16, le temps d'exécution beaucoup plus court permet de réduire fortement l'énergie totale consommée pour effectuer le calcul. À l'inverse, le float64 présente la consommation énergétique la plus importante en raison de son temps d'exécution très élevé.



Enfin, les performances mesurées en TFLOPS montrent que le float16 permet d'atteindre un débit de calcul nettement supérieur aux autres formats. Ces résultats mettent donc en évidence l'intérêt des formats réduits dans les calculs de deep learning et expliquent pourquoi des techniques comme la quantification sont largement utilisées dans les moteurs d'inférence modernes.

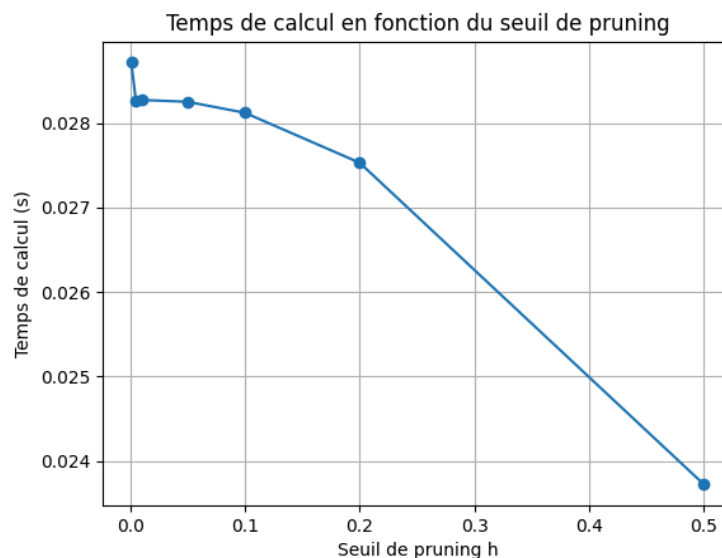


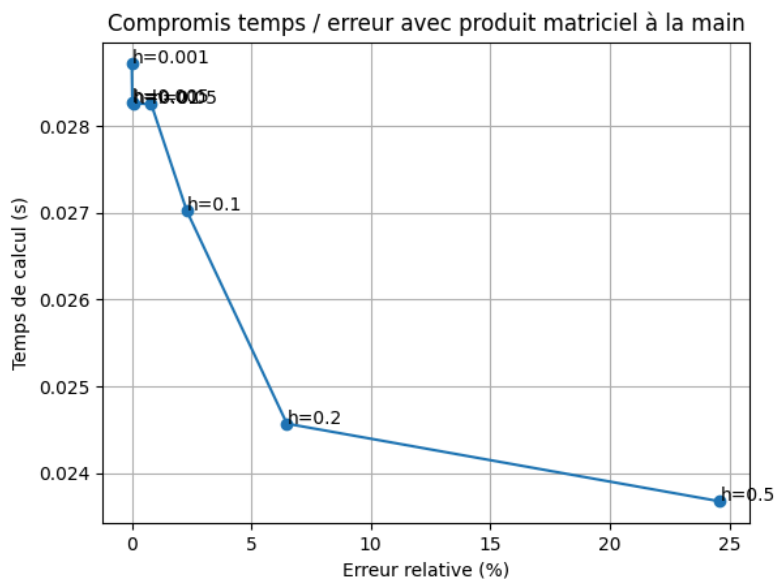
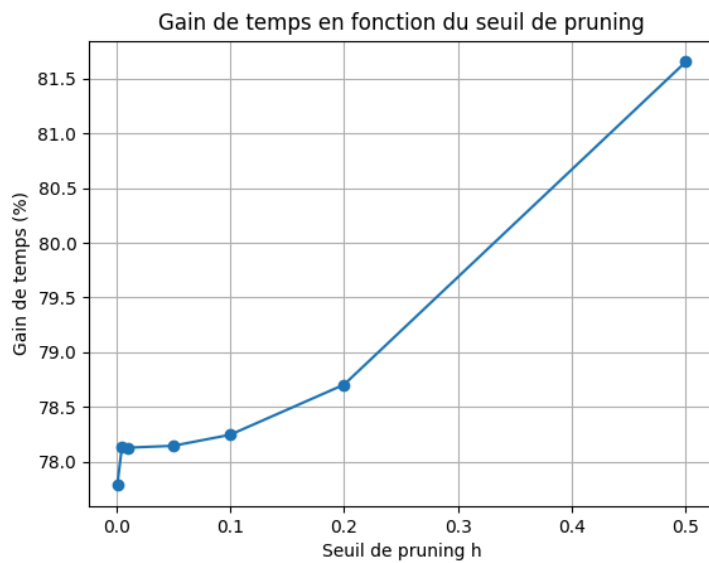
Des expérimentations complémentaires ont également été réalisées sur des formats quantifiés int8 et int4 afin d'étudier l'impact d'une réduction encore plus importante de la précision numérique. Ces tests ont principalement permis d'illustrer la diminution de la taille mémoire des matrices ainsi que les limites d'implémentations manuelles non optimisées pour les calculs entiers sur GPU. Les résultats détaillés ainsi que les graphes associés sont présentés en annexe.

Pruning;

Les résultats de la première méthode, en utilisant un seuil et le produit matriciel de python n'a pas été concluant, les temps de réponse étaient très proches. Les courbes sont visibles en annexe.

Le codage manuel du produit matriciel, en évitant de calculer les produits nuls, a apporté une large amélioration du temps de réponse, sans pour autant affecter la précision du modèle, surtout pour les petites valeurs du seuil de pruning h .

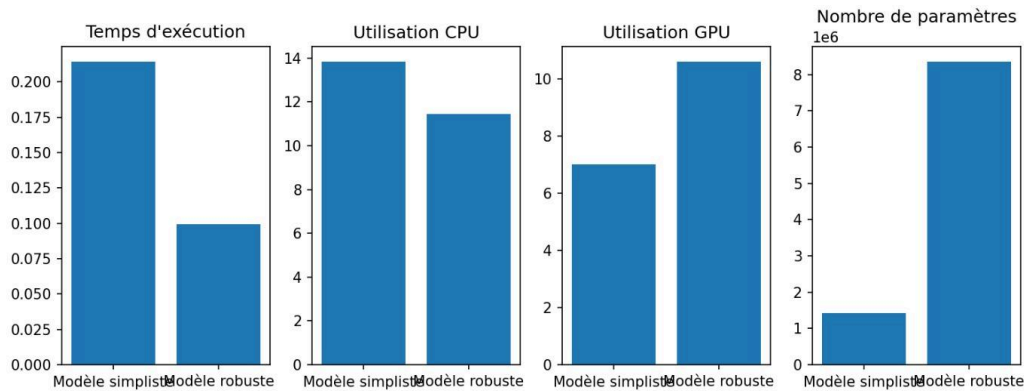




La première courbe montre que le pruning fonctionne et que l'on gagne au moins 78% du temps par rapport au calcul avant l'opération de pruning. Cependant, comme l'erreur augmente proportionnellement au seuil de pruning, le meilleur seuil n'est pas forcément celui qui nous fait gagner le plus de temps.

On peut l'observer sur la dernière figure où le meilleur compromis temps/erreur est pour un petit h , jusqu'à $h=0.2$, après cette valeur, nous perdons trop d'informations dans l'opération pruning malgré le temps gagné.

Optimisation de l'architecture du réseau de neurones



modèle simple vs modèle robuste

J'ai voulu d'abord tester la différence de consommation / temps d'exécution pour un modèle simpliste de segmentation d'image et un modèle plus robuste avec l'ajout de Batch Norm et de Max Pooling layers. On s'attend à un temps d'exécution plus faible pour le modèle robuste au détriment de la consommation. Bien que cette tendance se vérifie pour le GPU, on remarque que c'est l'inverse pour le CPU.

Après quelques recherches j'ai supposé que certains layer comme le Batch Norm augmentent la consommation du GPU pour optimiser l'architecture et ainsi consommer moins sur le CPU mais je ne comprends pas exactement le mécanisme.

Les résultats sur les différentes versions de YOLO sont présentés dans la partie "Résultats du groupe" ci-dessous.

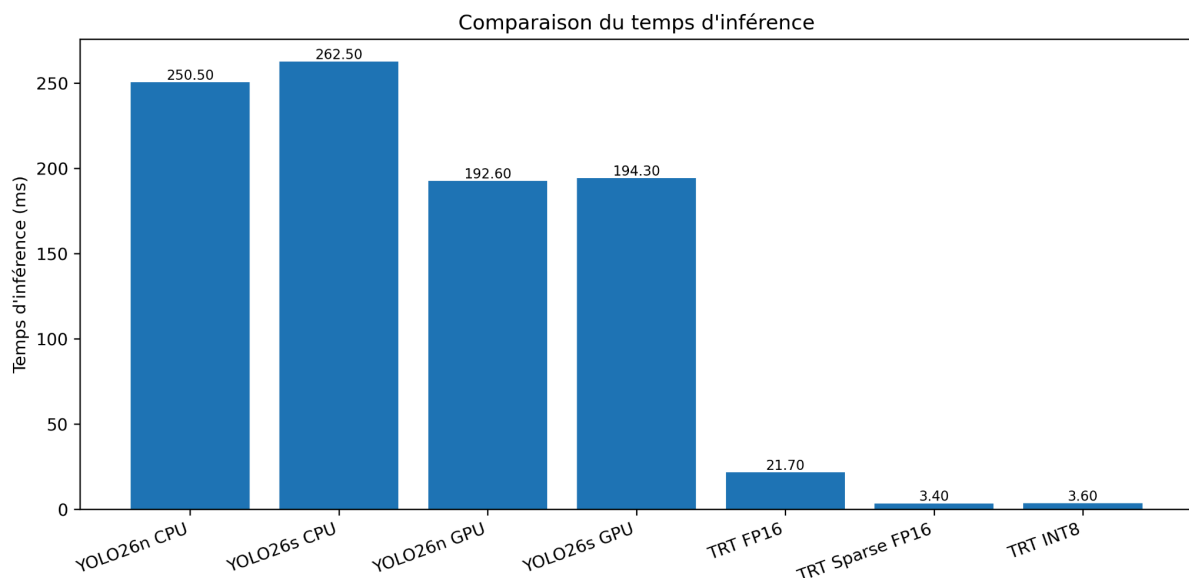
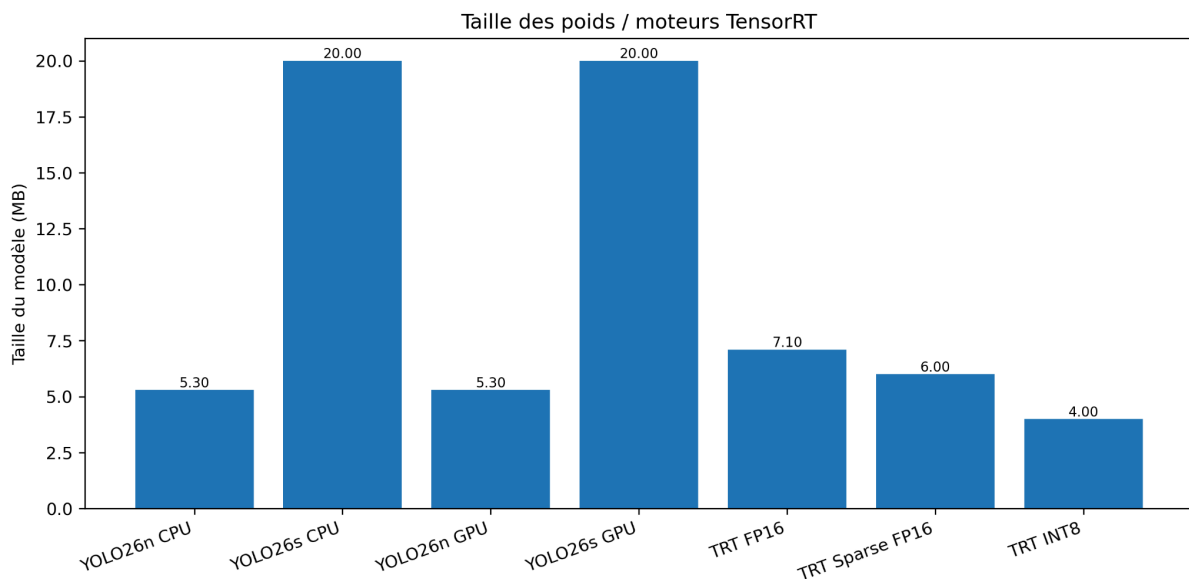
VI.2 Résultats du groupe

Afin d'évaluer l'impact des différentes optimisations étudiées au cours du projet, plusieurs versions du modèle YOLO26 ont été comparées : YOLO26n et YOLO26s sur CPU et GPU, ainsi que des versions optimisées avec TensorRT en FP16, Sparse FP16 et INT8. Les expérimentations ont porté sur plusieurs critères tels que le temps d'inférence, le débit de traitement, la taille du modèle, l'utilisation des ressources matérielles ainsi que la consommation énergétique.

Les résultats montrent d'abord que les versions embarquées de YOLO, comme YOLO26n, présentent un gain de taille important par rapport à des versions plus lourdes comme YOLO26s. Ce gain est intéressant pour un déploiement sur système embarqué, où la mémoire disponible est limitée. Cependant, on observe que certaines versions optimisées avec TensorRT peuvent présenter une légère augmentation de taille par rapport au modèle YOLO26n initial, notamment en raison des fichiers générés lors de la conversion et de la quantification.

Les optimisations TensorRT apportent également un gain significatif en temps d'inférence et en débit de traitement. Les versions optimisées atteignent un nombre d'images par seconde largement supérieur aux versions CPU et GPU classiques, tout en conservant une qualité de détection satisfaisante dans les tests réalisés. Les résultats détaillés concernant les FPS sont présentés en annexe.

Ces premiers résultats montrent que l'optimisation ne se limite pas uniquement à la réduction de la taille du modèle, mais concerne également le compromis entre mémoire, rapidité d'exécution, consommation énergétique et qualité de détection.



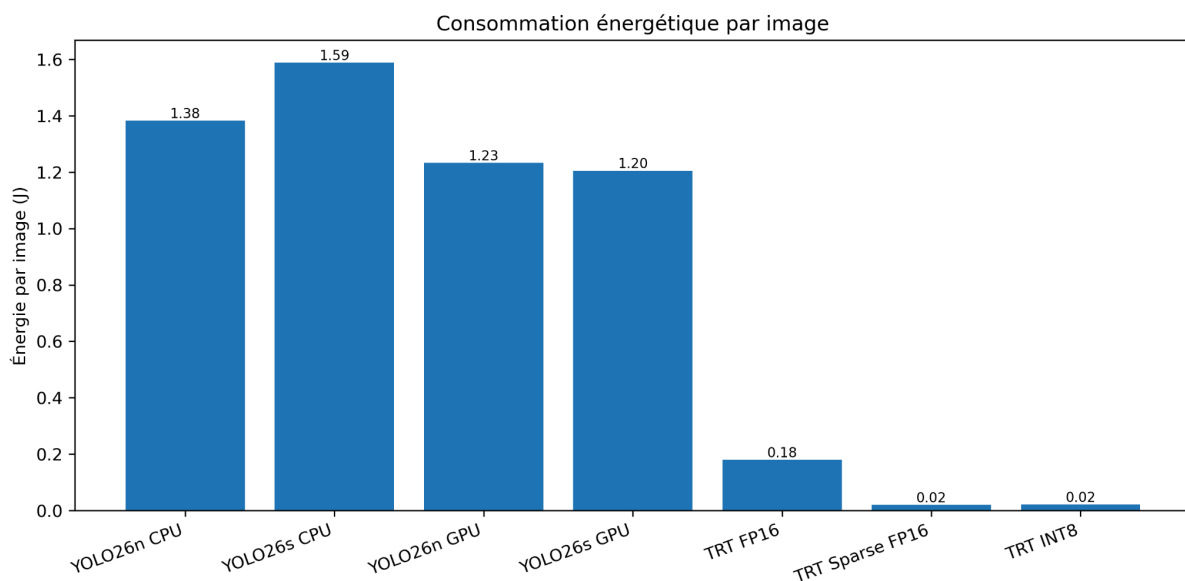
VI.3 Consommation énergétique

La consommation énergétique constitue l'un des critères les plus importants pour le déploiement de modèles d'intelligence artificielle sur des systèmes embarqués. Afin d'évaluer l'efficacité des différentes optimisations étudiées, l'énergie consommée par image a été mesurée pour plusieurs versions du modèle YOLO26.

Les résultats montrent une réduction importante de l'énergie consommée grâce aux optimisations TensorRT. Alors que les versions exécutées sur CPU consomment environ 1,4 à 1,6 J par image, la version TensorRT FP16 réduit cette consommation à environ 0,18 J par image. La version TensorRT INT8 obtient les meilleurs résultats avec une consommation proche de 0,02 J par image.

Ces résultats montrent que les techniques de quantification et les optimisations TensorRT permettent de diminuer fortement le coût énergétique de l'inférence. Cette réduction est principalement due à la diminution du temps d'exécution, qui compense largement la puissance consommée par le matériel pendant les calculs.

L'amélioration observée confirme l'intérêt de ces techniques pour le déploiement de modèles de vision par ordinateur sur des plateformes embarquées, où l'autonomie énergétique constitue souvent une contrainte majeure.



VI.4 Impact sur les performances

L'impact des différentes optimisations sur les performances du modèle a été évalué à travers sa capacité à détecter correctement les objets présents dans les images de test. Les différentes versions de YOLO26 ont été appliquées à des images contenant notamment des bus et des visages afin de vérifier que les optimisations n'altéraient pas la qualité des prédictions.

Les résultats obtenus montrent que les versions CPU, GPU, TensorRT FP16 et TensorRT Sparse FP16 conservent une qualité de détection satisfaisante. Les objets présents dans les images sont correctement identifiés malgré les optimisations appliquées au modèle.

En revanche, la version TensorRT INT8 présente une légère dégradation des performances de détection. Cette perte de précision s'explique probablement par la quantification plus agressive du modèle, qui réduit davantage la précision numérique des calculs.

Les résultats montrent ainsi qu'il est possible de réduire fortement le temps d'inférence et la consommation énergétique sans dégrader significativement la qualité de détection. Parmi les configurations testées, la version **TensorRT Sparse FP16** apparaît comme le meilleur compromis entre performances, qualité de détection et efficacité énergétique.

VI.5 Analyse et interprétation des résultats

Les résultats obtenus montrent que l'efficacité des techniques d'optimisation dépend fortement du contexte d'utilisation. Les expérimentations individuelles menées sur le pruning appliqué à des produits matriciels n'ont pas permis d'obtenir les gains de performances espérés sur GPU. Malgré l'augmentation de la sparsité, les architectures utilisées restent fortement optimisées pour les calculs denses, ce qui limite l'impact du pruning lorsqu'il est appliqué seul.

Cependant, les résultats obtenus sur le modèle YOLO26 montrent qu'une approche combinant plusieurs optimisations peut s'avérer particulièrement efficace. Les versions TensorRT FP16, TensorRT Sparse FP16 et TensorRT INT8 présentent toutes des améliorations importantes par rapport aux versions CPU et GPU classiques, tant en termes de temps d'inférence que de consommation énergétique.

Une comparaison entre YOLO26s et YOLO26n met également en évidence l'intérêt des architectures légères pour les systèmes embarqués. Les résultats obtenus sur le CPU ARM de la Jetson montrent que YOLO26n offre de meilleures performances que YOLO26s, avec des temps d'inférence plus faibles tout en conservant des capacités de détection satisfaisantes. Cet avantage est cependant beaucoup moins marqué lors de l'exécution sur le GPU de la Jetson. En effet, bien que la Jetson soit une plateforme embarquée, elle dispose d'un GPU NVIDIA dont l'architecture se rapproche davantage de celles utilisées sur des ordinateurs conventionnels. La puissance de calcul disponible compense alors en partie la différence de complexité entre les deux modèles. Cette observation confirme que les architectures compactes comme YOLO26n sont particulièrement avantageuses lorsque les ressources de calcul sont limitées, tandis que leur bénéfice est moins visible sur des plateformes embarquées disposant d'une accélération GPU performante.

Parmi les configurations testées, TensorRT Sparse FP16 apparaît comme la solution la plus intéressante. Cette version atteint une efficacité énergétique comparable, voire supérieure, à celle obtenue avec la quantification INT8, tout en conservant les avantages du format FP16 en termes de précision. À l'inverse, la version INT8 présente une légère dégradation de la qualité de détection malgré ses excellentes performances.

Ces résultats montrent qu'il est possible d'améliorer fortement l'efficacité énergétique d'un modèle d'intelligence artificielle sans nécessairement recourir aux niveaux de quantification les plus agressifs. L'association du pruning et des optimisations TensorRT constitue ainsi une solution particulièrement pertinente pour le déploiement de modèles de vision par ordinateur sur des systèmes embarqués.

** En ce qui concerne l'utilisation mémoire (RAM et SWAP) une conclusion est disponible en annexe (Figures 4 et 5)

VII- Limites et difficultés rencontrées

Au cours du projet, plusieurs difficultés techniques et méthodologiques ont été rencontrées lors de l'implémentation et de l'évaluation des différentes méthodes d'optimisation.

Concernant l'optimisation de l'architecture des modèles, les travaux réalisés ont principalement conduit au choix du modèle YOLO26n comme architecture adaptée à une utilisation embarquée. Cependant, il n'a pas été possible d'affiner davantage cette architecture dans le temps imparti au projet.

Les expérimentations liées au pruning n'ont pas permis d'obtenir une réduction significative du temps de traitement malgré plusieurs essais. Une des hypothèses avancées est que certaines opérations GPU utilisées ne prennent pas pleinement en compte les valeurs nulles générées par la sparsité, limitant ainsi les gains de performance attendus. Les architectures GPU modernes étant fortement optimisées pour les calculs denses, certaines méthodes de sparsité peuvent ne pas apporter les améliorations espérées dans certains contextes.

Des difficultés importantes ont également été rencontrées lors de la mise en œuvre de l'environnement de calcul GPU sur la plateforme Jetson Orin Nano. L'installation et la configuration des bibliothèques CUDA, cuDNN et PyTorch se sont révélées complexes en raison de problèmes de compatibilité entre certaines versions logicielles. Plusieurs erreurs liées au chargement de bibliothèques système, à l'initialisation des composants CUDA ainsi qu'à l'allocation mémoire GPU ont nécessité de nombreuses phases de diagnostic et de correction avant de pouvoir réaliser les expérimentations. Ces difficultés ont fortement mobilisé une partie du temps du projet et ont limité le nombre de tests pouvant être effectués.

Par ailleurs, les mesures énergétiques réalisées restent des estimations basées sur la puissance délivrée par le matériel pendant l'exécution des modèles. Une analyse plus précise nécessiterait des outils matériels spécialisés permettant de mesurer directement la consommation énergétique réelle du système.

Le projet a également été confronté à des contraintes de temps importantes. Certaines optimisations nécessitent de nombreuses phases de test et de comparaison afin d'obtenir des résultats réellement exploitables. Le temps disponible n'a donc pas permis d'explorer l'ensemble des techniques d'optimisation envisageables.

Enfin, les expérimentations ont été réalisées sur un nombre limité de modèles et de configurations matérielles. Les résultats obtenus ne peuvent donc pas être généralisés à l'ensemble des architectures d'intelligence artificielle ou des plateformes embarquées existantes. Des expérimentations supplémentaires seraient nécessaires afin de confirmer les tendances observées au cours du projet.

VIII- Perspectives d'amélioration

Plusieurs perspectives d'amélioration peuvent être envisagées afin de prolonger les travaux réalisés dans le cadre du projet Cassiopée.

Une première piste consiste à approfondir l'optimisation des modèles sur plateforme embarquée, notamment sur Jetson Orin Nano. Des tests plus complets permettraient d'évaluer plus précisément le comportement des différentes méthodes d'optimisation dans un contexte réel d'exécution embarquée.

L'utilisation d'outils spécialisés comme TensorRT pourrait également permettre d'améliorer davantage les performances des modèles. Ces outils proposent des optimisations avancées adaptées aux architectures GPU NVIDIA et peuvent réduire le temps d'inférence ainsi que la consommation énergétique.

Il serait également intéressant d'étendre les expérimentations à d'autres architectures de réseaux de neurones ou à des modèles plus complexes afin de comparer le comportement des différentes techniques d'optimisation selon les cas d'usage. Des modèles spécialement conçus pour l'embarqué, comme certaines variantes légères de YOLO ou MobileNet, pourraient notamment être étudiés plus en détail.

Une autre perspective concerne l'amélioration des mesures énergétiques. L'utilisation d'outils matériels spécialisés permettrait d'obtenir des données plus précises sur la consommation réelle des différents composants du système pendant l'exécution des modèles.

Enfin, les travaux réalisés dans ce projet pourraient être appliqués à d'autres domaines nécessitant une intelligence artificielle embarquée temps réel, comme la robotique, les drones autonomes, les véhicules intelligents ou les systèmes de surveillance embarqués.

IX- Conclusion

Le projet Cassiopée avait pour objectif d'étudier différentes méthodes permettant de réduire la consommation énergétique de modèles d'intelligence artificielle destinés à des systèmes embarqués. Dans un contexte où les modèles de deep learning deviennent de plus en plus coûteux en ressources, l'optimisation énergétique représente un enjeu important pour le développement d'applications embarquées temps réel.

Au cours du projet, plusieurs approches ont été étudiées, notamment la quantification, le pruning ainsi que l'optimisation de l'architecture des réseaux de neurones. Les expérimentations réalisées ont permis d'analyser l'impact de ces méthodes sur le temps d'exécution, l'utilisation mémoire, les performances des modèles et la consommation énergétique.

Les résultats obtenus ont montré que certaines techniques d'optimisation peuvent réduire le coût computationnel des modèles, mais également que les gains obtenus dépendent fortement de l'architecture matérielle utilisée et des mécanismes d'exécution GPU. Certaines limites liées à la gestion de la sparsité ou à la compatibilité matérielle ont notamment été mises en évidence au cours du projet.

Enfin, ce travail ouvre plusieurs perspectives d'amélioration, notamment autour de l'optimisation sur plateforme embarquée Jetson Orin Nano et de l'utilisation d'outils spécialisés. Ces travaux pourraient contribuer au développement de solutions d'intelligence artificielle plus adaptées aux contraintes énergétiques des systèmes embarqués modernes.

X-Annexes

Annexes 1: courbes sur la quantification

Les expérimentations réalisées sur les formats quantifiés int8 et int4 ont principalement permis d'illustrer l'impact de la réduction du nombre de bits sur la taille mémoire des matrices. Comme attendu, l'utilisation de formats entiers réduit fortement la mémoire nécessaire pour stocker les poids et les données manipulées lors du calcul.

Cependant, les résultats obtenus en temps d'exécution et en consommation énergétique restent ici moins performants que ceux observés avec le float16. Cela s'explique par le fait que les implémentations utilisées pour les calculs int8 et int4 étaient des implémentations manuelles non optimisées, contrairement au float16 qui bénéficie directement des bibliothèques CUDA et des kernels GPU optimisés. Ces résultats étant principalement illustratifs et ne représentant pas des implémentations industrielles optimisées, cette partie ne sera pas davantage approfondie dans la suite de l'étude.

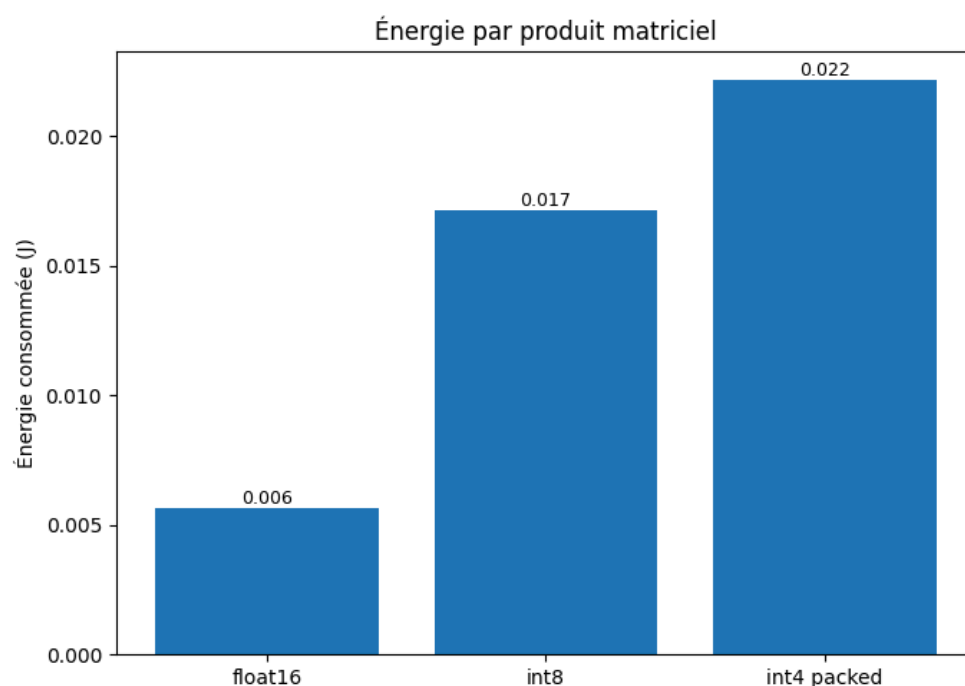


Figure 1 — Énergie consommée par produit matriciel pour différents formats quantifiés

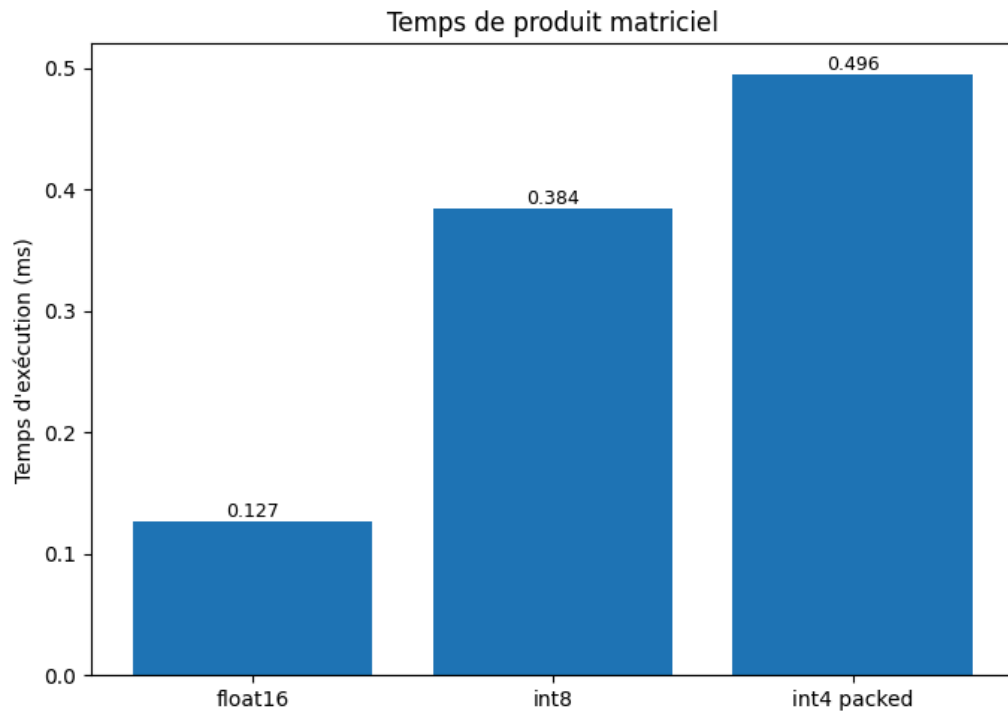


Figure 2 — Temps d'exécution du produit matriciel selon le niveau de quantification (float16, int8 et int4)

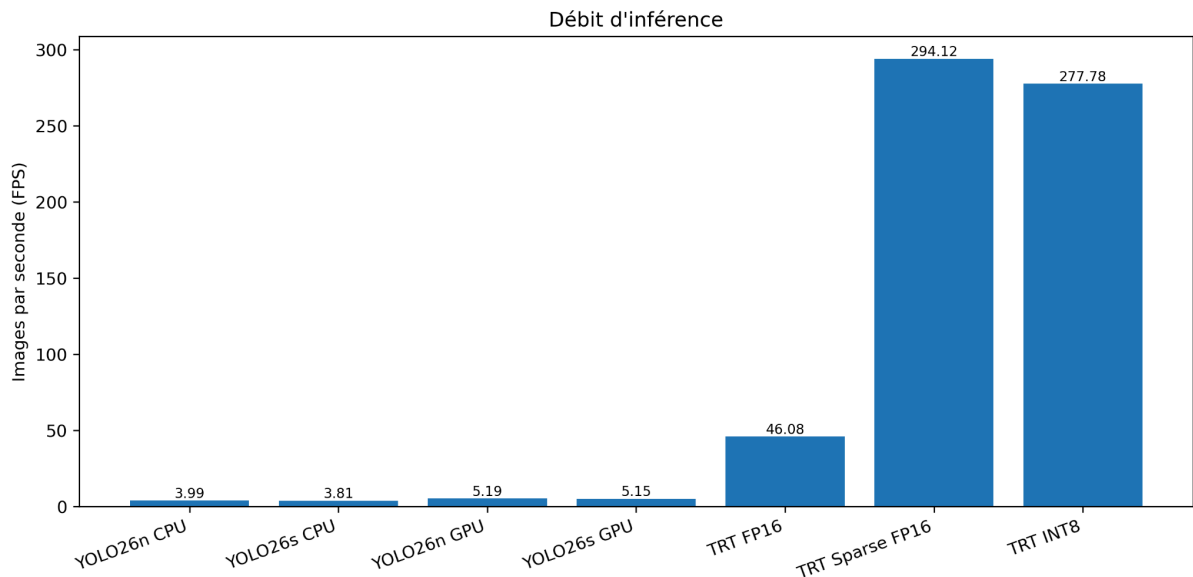


Figure 3 – Comparaison du débit d'inférence (FPS) des différentes versions de YOLO26

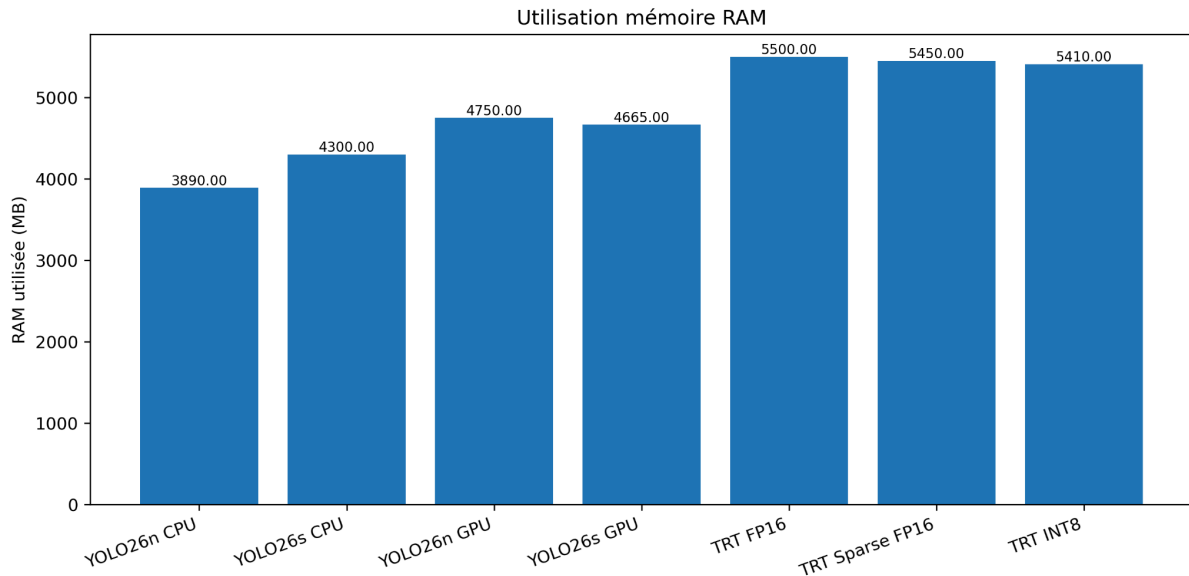


Figure 4 – Comparaison de l'utilisation de la mémoire vive (RAM) des différentes versions de YOLO26

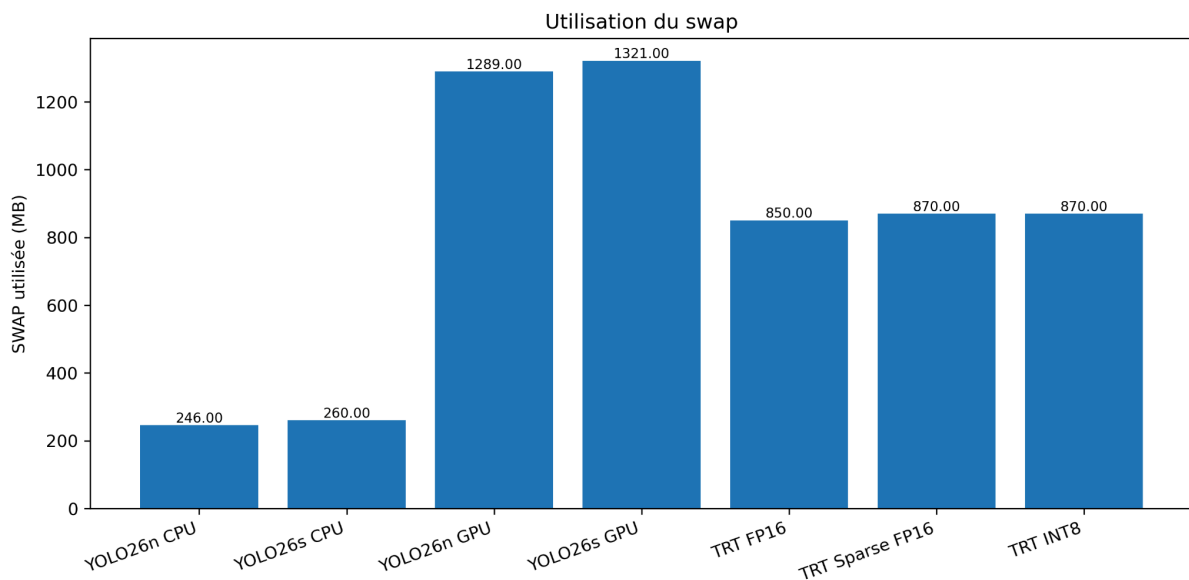


Figure 5 – Comparaison de l'utilisation de la mémoire SWAP pour les différentes configurations d'exécution de YOLO26 (CPU, GPU et TensorRT).

** Les résultats montrent que les exécutions sur CPU ARM utilisent significativement moins de mémoire SWAP que les exécutions sur GPU. Cette différence s'explique principalement par le fait que l'utilisation du GPU nécessite l'allocation de buffers supplémentaires pour le transfert des données, le stockage intermédiaire des tenseurs et l'exécution parallèle des opérations. Lorsque la mémoire vive disponible devient insuffisante, une partie des données est déplacée vers le SWAP, ce qui augmente son utilisation. À l'inverse, l'exécution sur CPU mobilise moins de mémoire auxiliaire et présente donc une empreinte mémoire globale plus faible. On observe également que les optimisations TensorRT permettent de réduire l'utilisation du SWAP par rapport aux versions GPU classiques, grâce à une meilleure

gestion des ressources mémoire et à l'optimisation des calculs lors de l'inférence.

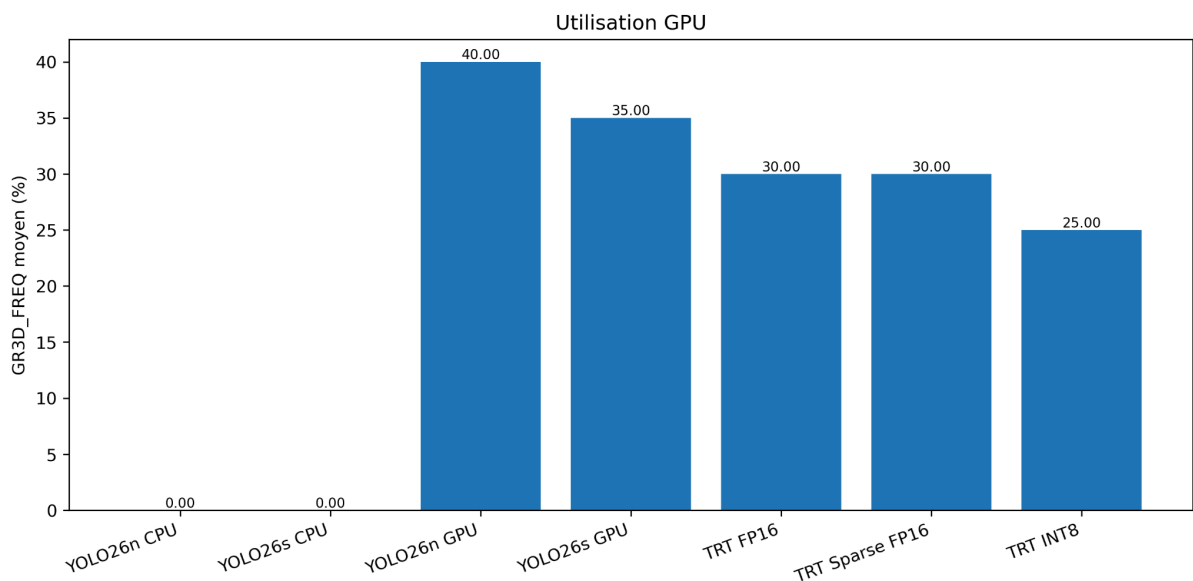
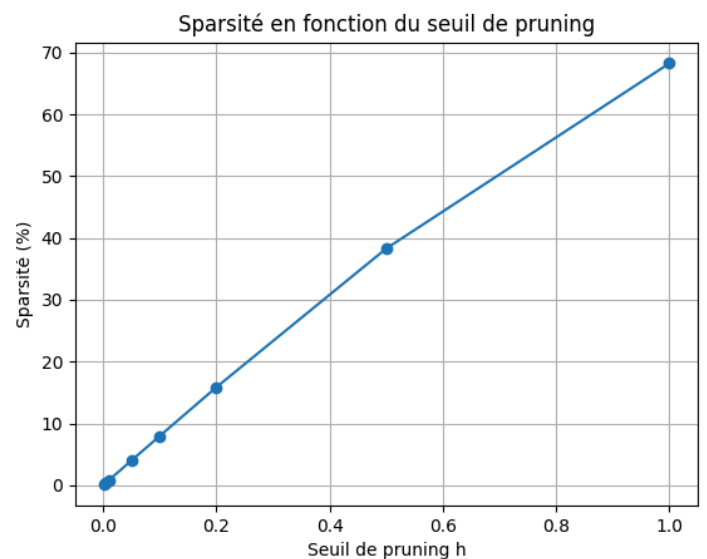
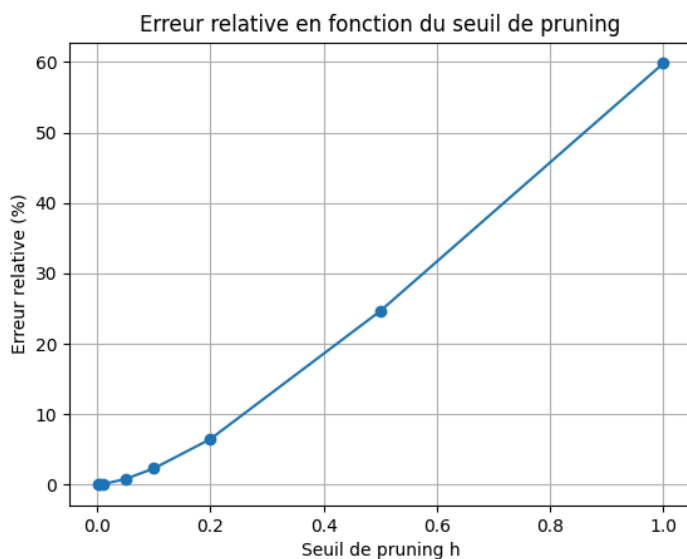


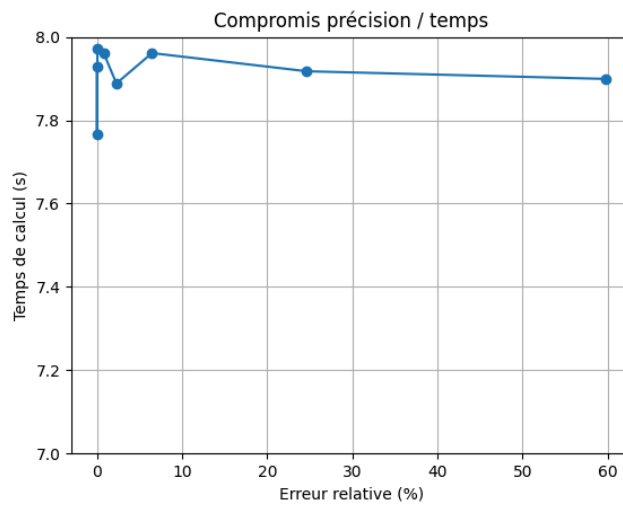
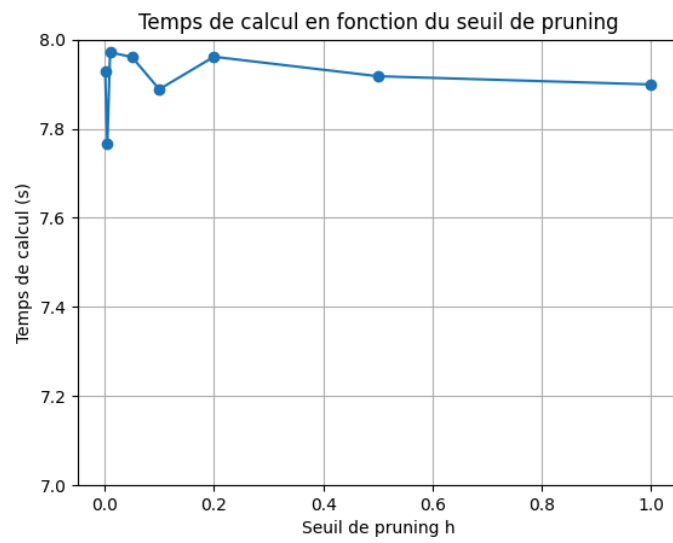
Figure 6 – Comparaison de l'utilisation de la GPU pour les différentes configurations d'exécution de YOLO26 (CPU, GPU et TensorRT)

Annexes 2: courbes sur le pruning

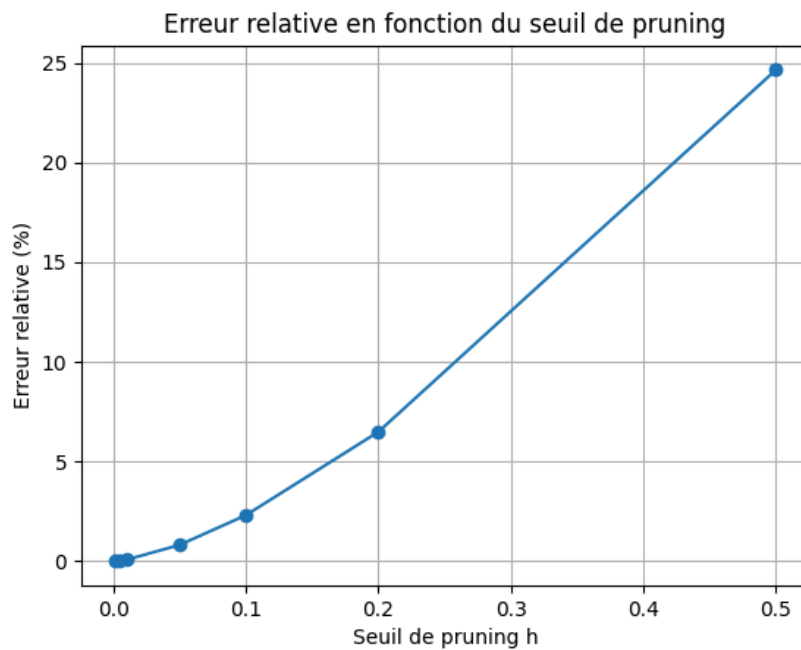
1) Voici les courbes obtenues sur ma première idée:

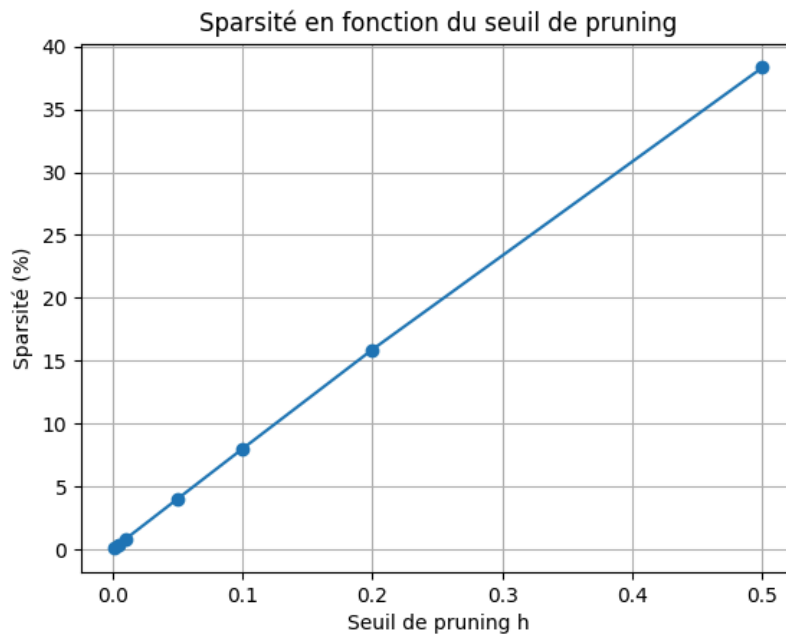
Ces courbes témoignent du bon fonctionnement de mon pruning, la sparsité augmente, cependant les courbes temporelles montrent que le temps reste inchangé, environ 1% de gain de temps



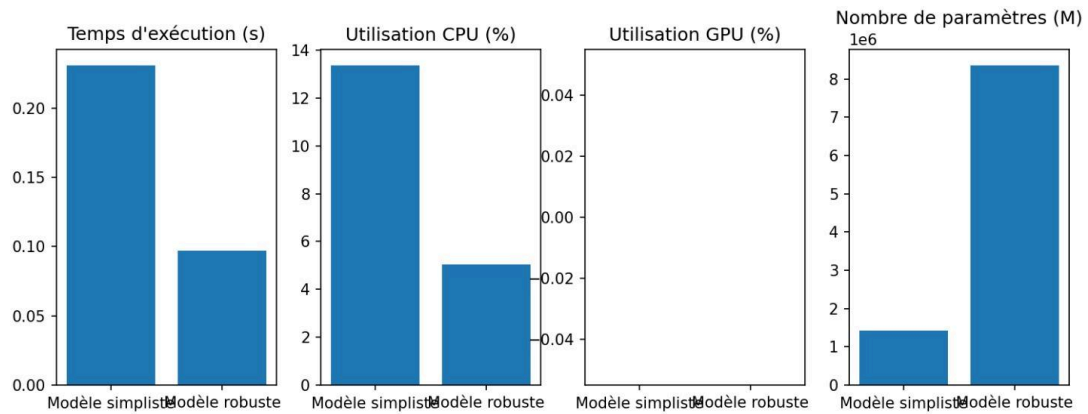


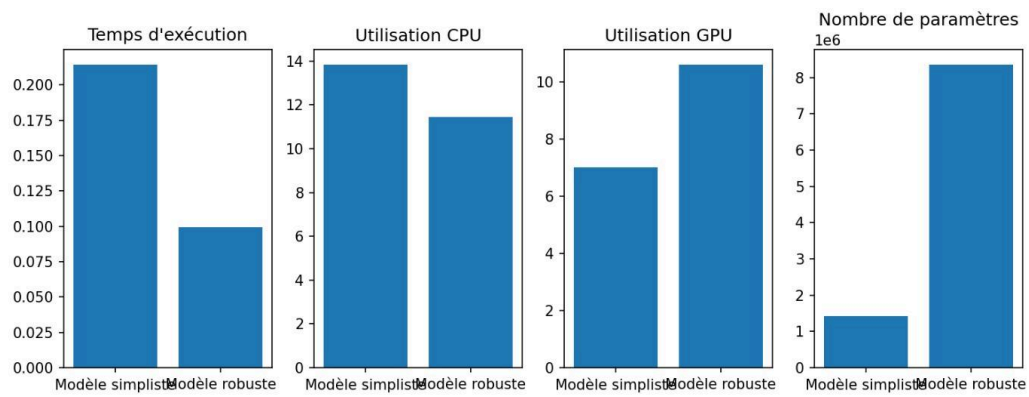
2) Voici le reste des courbes sur le pruning réussi:





Annexes 3: courbes sur l'optimisation du modèle.





Voici un exemple de courbes obtenues lors du travail sur le modèle simple et le modèle robuste. Ces deux mesures ont été réalisées successivement dans les mêmes conditions. On remarque qu'on obtient des résultats pour le GPU dans un cas mais pas dans l'autre. Ces problèmes de détection du GPU ont été récurrents tout au long du projet.

XI- Bibliographie

<https://www.ultralytics.com/yolo>

<https://nvdam.widen.net/s/zkfqjmtds2/jetson-orin-datasheet-nano-developer-kit-3575392-r2>

<https://www.youtube.com/watch?v=v1oHf1KV6kM>

<https://docs.nvidia.com/deeplearning/tensorrt/latest/index.html>

<https://docs.nvidia.com/cuda/>

<https://docs.cupy.dev/en/stable/>

<https://arxiv.org/abs/1510.00149>

<https://www.youtube.com/watch?v=x57mjP37c7o>

<https://forums.developer.nvidia.com/>

https://www.reddit.com/r/MachineLearning/comments/l64ji8/d_has_anyone_here_gotten_any_advantage_in/

utilisation d'IA génératives comme aide pour les codes en python notamment pour l'initialisation des environnement